CMPT 276 Group Project Report

Bank Heist (Group 1)

Group Members: Mohamad Osman, Brian Chan, Keith, Daniil Bugakov, Dev Tomar

1 Overview

Bank Heist is a grid-based chase and collection game where the player controls a thief moving through a maze-like board to collect rewards while avoiding a guard. The goal is to collect all the cash on the map and escape through the exit without getting caught by the enemy (Guard) or losing all your balance (obstacles).

The game combines movement, hazards, rewards, and time tracking to create a simple but strategic gameplay experience. It is built using object-oriented design, with game logic, UI, and entities kept modular and separate, making the code easier to manage.

2 Core Gameplay Mechanics

The player controls the thief using directional movement across a grid. Movement is limited by walls, boundaries, and other occupied tiles, so the player must navigate carefully. At the same time, a guard is constantly moving toward the thief, thus making the game more challenging.

The game ends if the thief is caught by the guard or if the player's balance becomes negative. To win, the player must collect all the money and reach the exit.

3 Game Elements and Interactions

All items in the game are based on a common `Interactable` class, which ensures consistent behavior when collected. There are different types of items in the game. `CashStacks` increase the player's balance and reduce the number of remaining objectives, while `RareDiamonds` give a larger bonus. On the other hand, hazards like `InkDarts` decrease the balance, and `GlueTraps` temporarily stop the player from moving.

Levels are created using the `BoardFactory`, which builds a game board from predefined layouts. This makes it easier to design and test levels, since the layout is separated from the main game logic.

The game also includes a time and scoring system handled by the `PlaytimeManager`. It tracks how long the player has been playing, even across pauses, and displays the final time at the end. The score updates depending on what the player collects or triggers.

4 User Interface and States

The game uses a state-based system to manage what is happening at any moment, such as the main menu, gameplay, pause screen, win screen, or game over screen. The UI is handled separately

by the `InterfaceRenderer`, which draws the correct screen and HUD elements depending on the current state.

Keeping the UI separate from game logic made the system cleaner and easier to maintain, since changes in visuals do not affect the core mechanics.

5 Software Design and Architecture

The project follows a modular object-oriented design. Common behavior is grouped into base classes like `MovingEntity` for characters and `Interactable` for items. This reduces repeated code and makes it easier to add new features.

Different parts of the system are clearly separated. For example, the board handles game logic, the UI handles rendering, and the time manager handles timing. The `BoardFactory` is used to generate levels, which helps keep the code organized and scalable.

6 Testing Strategy

We included several unit and integration tests to ensure everything works correctly. These tests cover movement, collisions, game state changes, item interactions, UI behavior, and timing.

The tests verify actions such as: guard catching the thief ends the game, the exit only works after all money is collected, and that the player cannot move through walls. Having these tests helped catch bugs early and made the system more reliable.

7 Deviations from Original Plan

From the initial design, most of the core ideas were kept the same, but some structural changes were made during implementation to simplify the system and improve modularity.

Instead of having multiple separate classes for handling game logic (such as game engine, input handling, and exit logic), these responsibilities were consolidated into the `Board` class to reduce complexity.

Additionally, the original design relied more on rigid structures like enums and switch statements, but this was refactored into a more flexible approach using abstraction (e.g., the `Interactable` system). New components such as `BoardFactory`, `ItemFactory`, `PlaytimeManager`, and dynamic diamond spawning were also introduced to improve organization and extend functionality.

8 Challenges Faced

One of the main challenges was managing interactions between multiple systems such as player movement, enemy behavior, items, and UI updates. Ensuring that all components worked together correctly required continuous testing and adjustments.

Another major challenge was refactoring the code as the project grew. Early designs led to too much logic being placed in single classes, which required restructuring into smaller, more focused

components. Implementing the guard's pathfinding behavior and integrating it with the board was also complex.

Finally, testing and debugging revealed several issues that required changes to the implementation, such as modifying methods to be testable and improving overall code structure.

9 Key Lessons Learned

This project reinforced the importance of separating different parts of a system. It also emphasized making regular commits on Git and maintaining constant communication within the team. Additionally, it showed how useful testing is for catching issues early.

The use of refactoring, along with design patterns such as abstraction and factories, made the code easier to extend, and a modular structure made teamwork much smoother.

10 Conclusion

Overall, Bank Heist is a well-structured and interactive game that demonstrates good software engineering practices. Compared to the initial plan, the final version is more complete, better organized, and more reliable due to improved design and testing.

The project shows how good structure and planning can lead to a maintainable and extensible system.

Video Link

Project Demo Video